BIT.MLMR

Summary

Artificial Intelligence
2025-2026







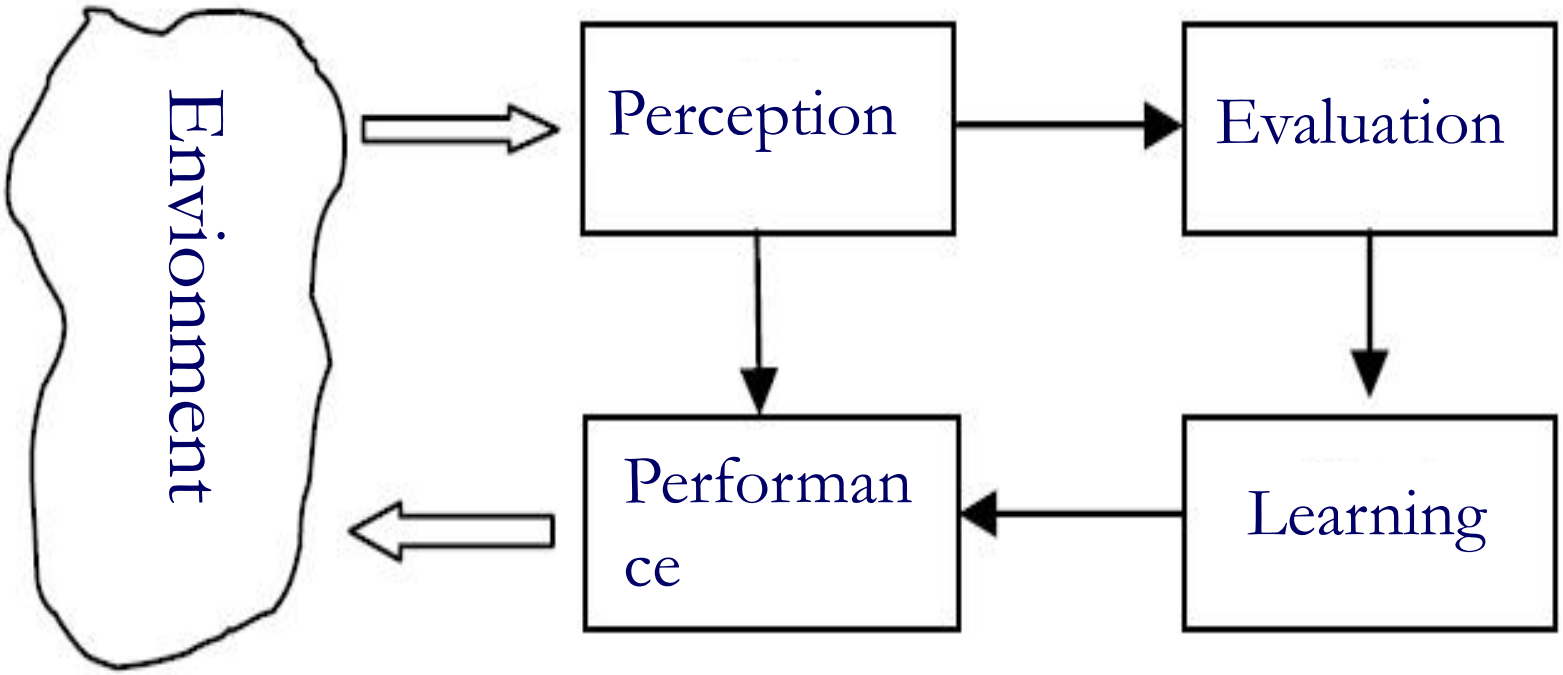


Understand and BUILD intelligent entities

Two Views: Weak AI (Turing Test); Strong AI (Chinese Room Objection)

Six Approaches: Symbolic AI; Connectionism; Machine Learning; Nouvelle AI; Simulated Evolution; Swarm Intelligence

The Architecture of intelligent system with learning capability



Lecture Outline

- **Road Map1: Learning**
  - Neural Network
  - Decision Tree
  - Reinforcement Learning
- **Road Map2: Search**
  - Deterministic Search
  - Evolutionary Computing
  - Swarm Intelligence
- **Road Map3: Reasoning**
  - First-Order Logic (Resolution)

Road Map 1: Learning

LEARNING DENOTES THE CHANGES

Defining the Learning Task

Improve on Task with respect to Performance metric based on Experience

Examples

T: Playing checkers
P: Percentage of games won against an arbitrary opponent
E: Playing practice games against itself

T: Recognizing hand-written words
P: Percentage of words correctly classified
E: Database of human-labeled images of handwritten words

T: Driving on four-lane highways using vision sensors
P: Average distance traveled before a human-judged error
E: A sequence of images and steering commands recorded while observing a human driver.

Types of Feedback for Learning

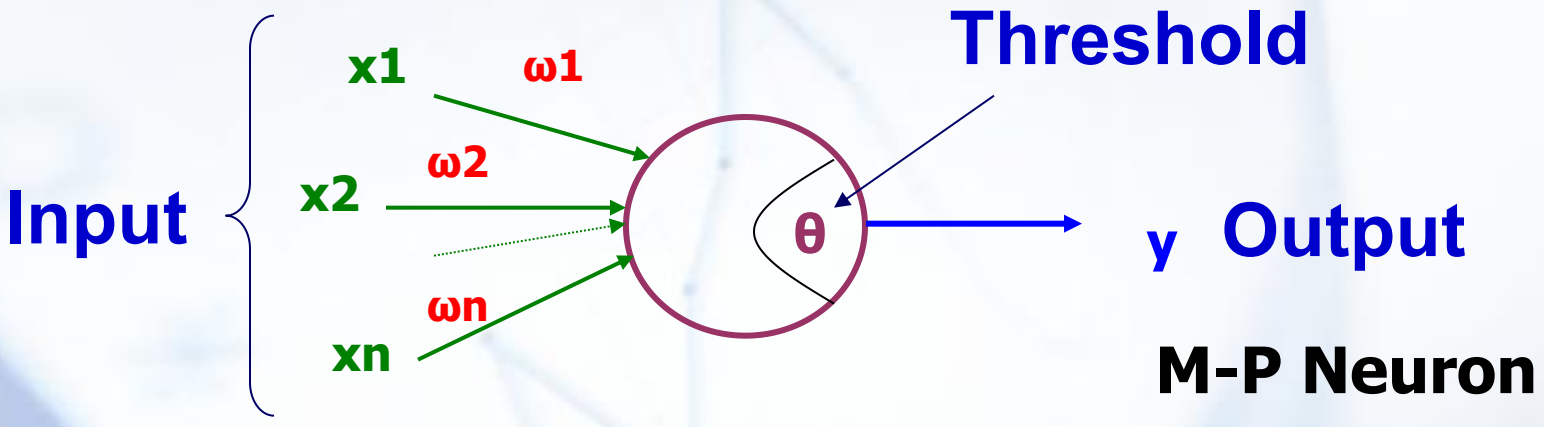
- **Supervised Learning**
  - (Input, output) pairs of the function to be learned can be perceived or are given.
- **Unsupervised Learning**
  - No information at all about given output
- **Reinforcement Learning**
  - in the case of the agent acts on its environment, it receives some evaluation of its action (reinforcement), but is not told of which action is the correct one to achieve its goal

The Essence of Supervised Learning

- **Function Estimation**
- **Three problems**
  1. How to assume the function form?
  2. How to design the learning objective?
  3. How to find the optimal machine? (search)

1.1 How to assume the function form?-1

❖ Neural Network



$$y = f(\sigma) = f\left(\sum_{i=1}^{n} \omega_i x_i - \theta\right)$$

Combination Function

Activation Function

Combination Functions

Weighted Sum

$$\xi = g(\boldsymbol{X}) = \sum_{i=1}^{n} \omega_i x_i - \theta$$

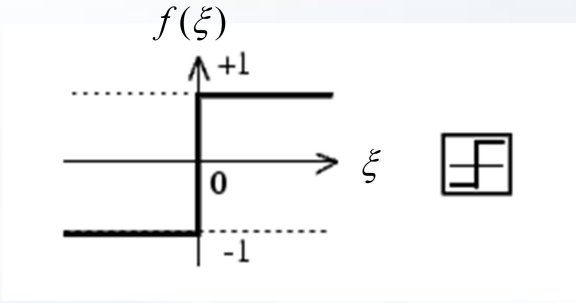
$$y = f(\xi) = f\left(\sum_{i=1}^{n} \omega_i x_i - \theta \right) \longrightarrow y = f(\xi) = f\left(\sum_{i=0}^{n} \omega_i x_i \right)$$

Radial Distance

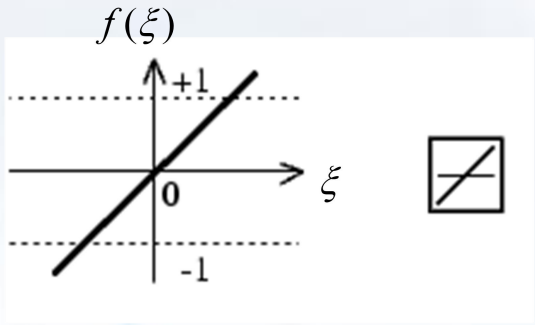
$$\xi = \|\boldsymbol{X} - \boldsymbol{C}\| = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

$$y = f(\xi) = f\left(\sqrt{\sum_{i=1}^{n} (x_i - c)^2} \right)$$

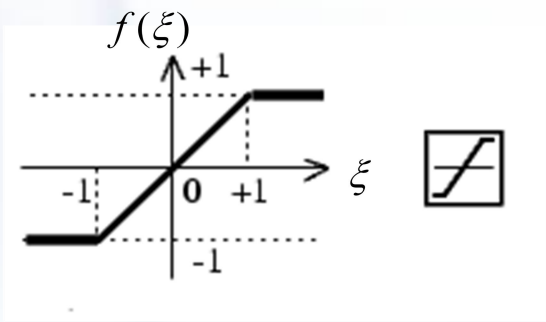
Activation Functions



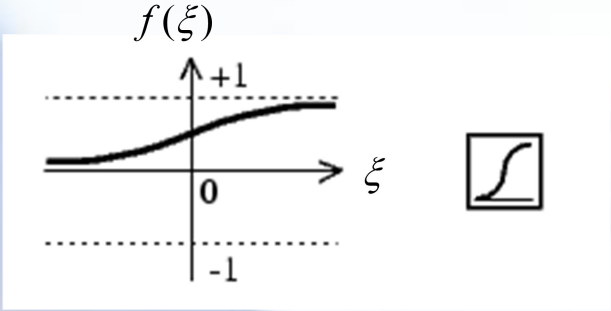
Threshold



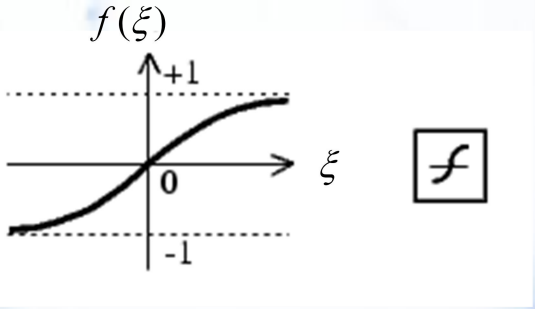
Linear



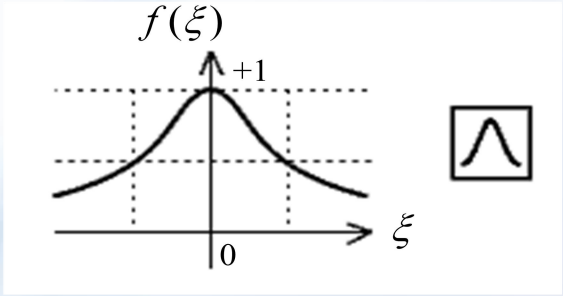
Saturating Linear



Logistic Sigmoid

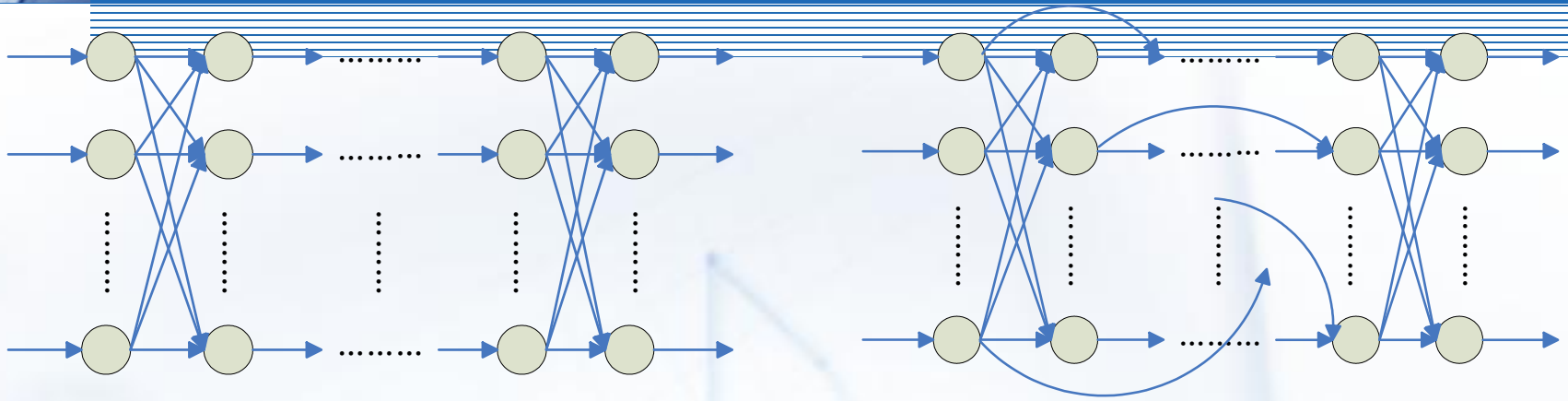


Hyperbolic tangent Sigmoid

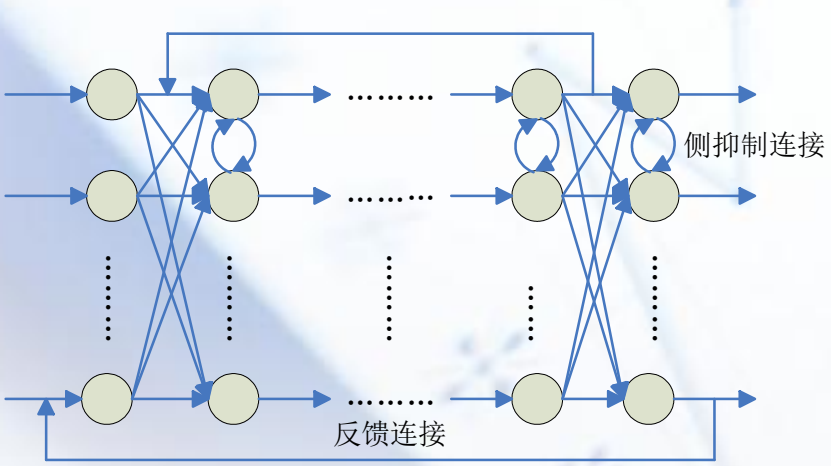


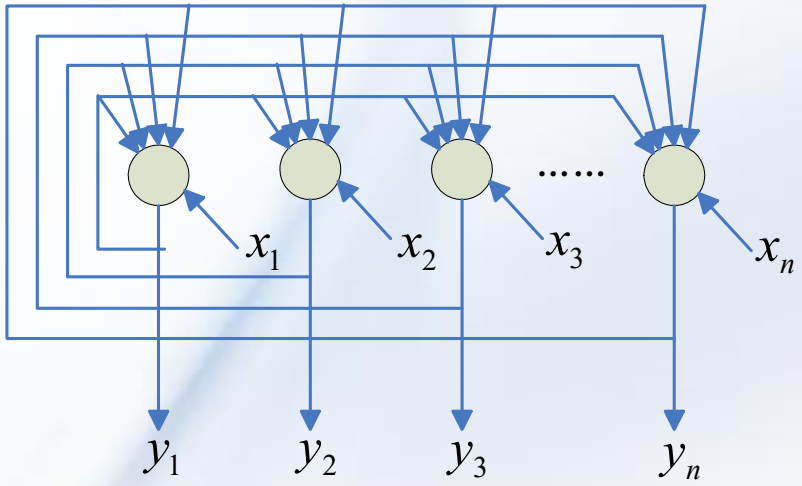
Gaussian

Architectures



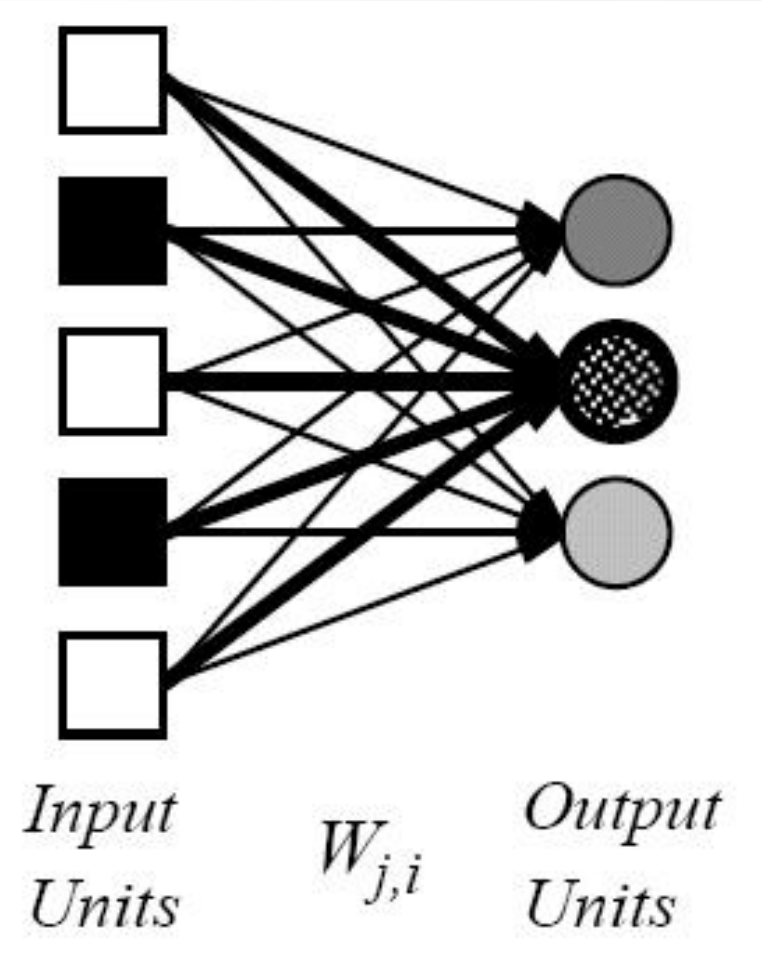
General structures of feedforward networks





General structures of feedback networks

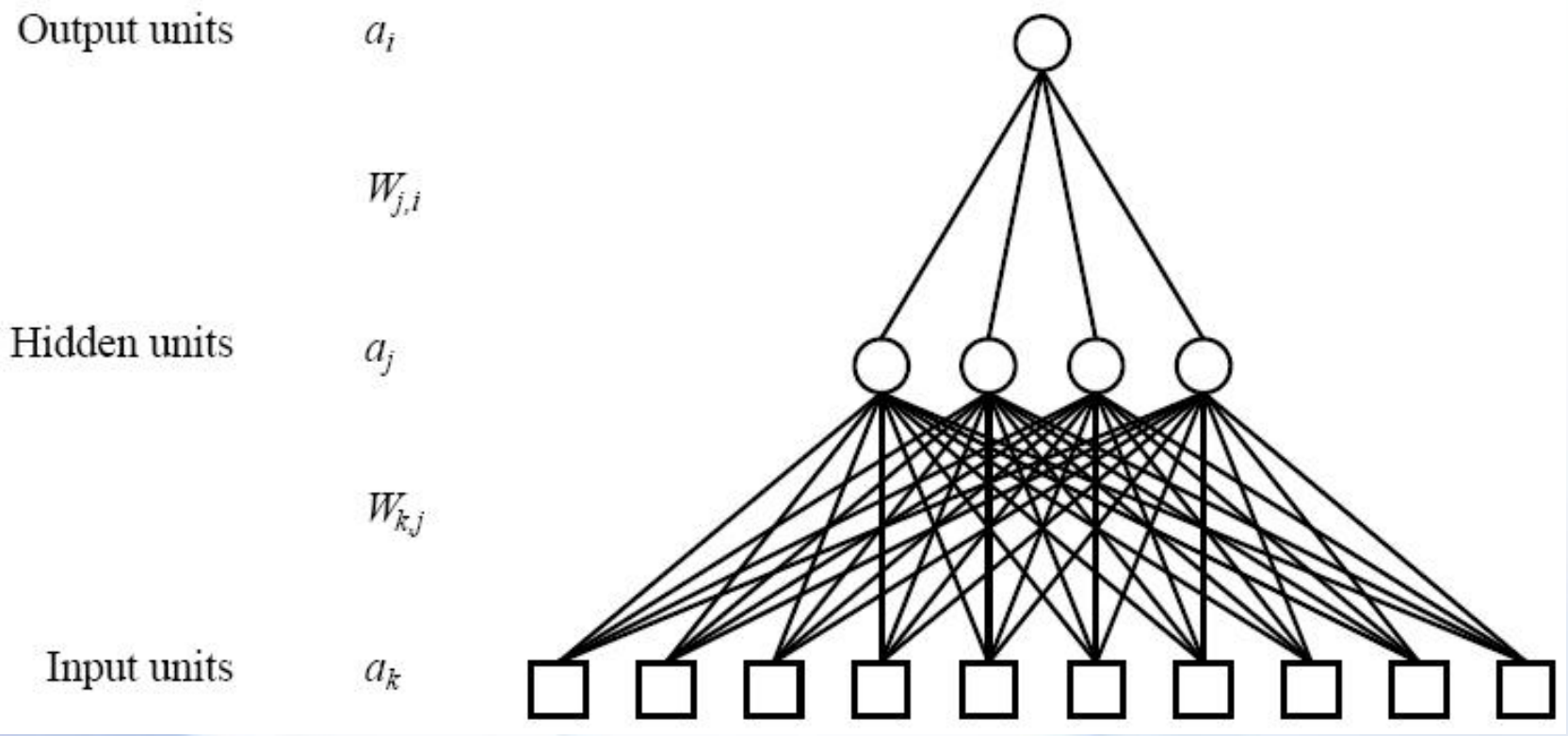
Single-layer Perceptron



- Binary Threshold Activation Function
- Output units all operate separately, i.e. no shared weights

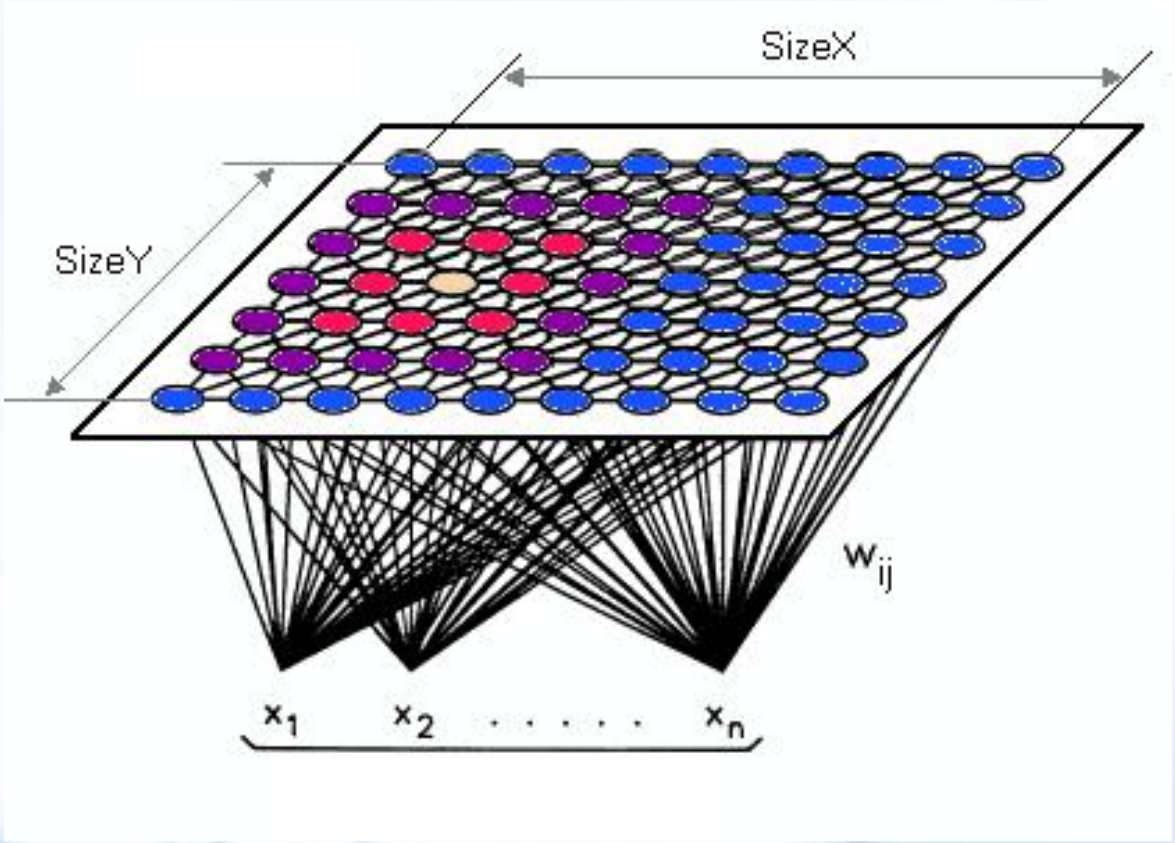
Rosenblatt, 1957

Multi-layer Perceptron (MLP)



- Layers are usually fully connected
- numbers of hidden units typically chosen by hand

Self-Organizing Feature Map (SOFM)

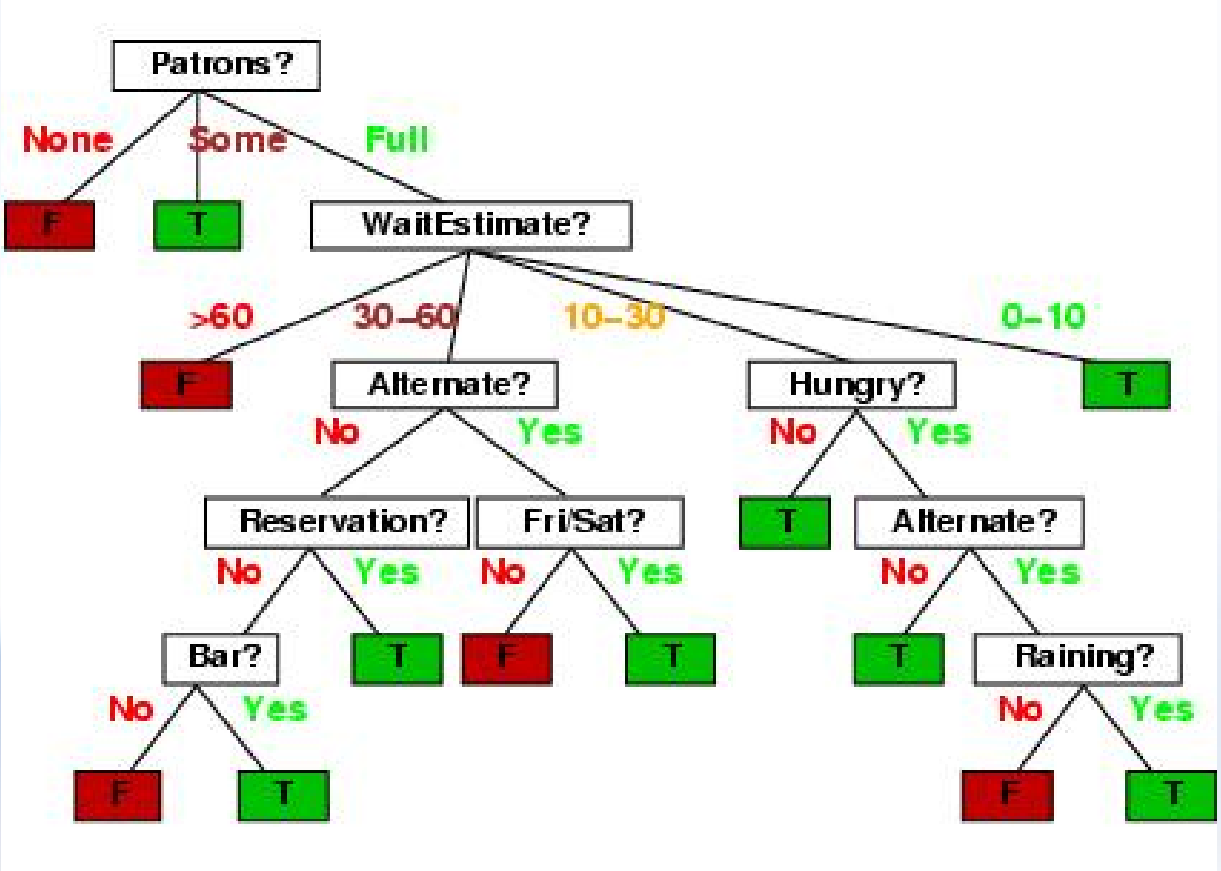


Picture：

http://www.lohninger.com/helpcsuite/kohonen_network_-_background_information.htm

How to assume the function form?-2

❖ Decision Tree



How to assume the function form?-3

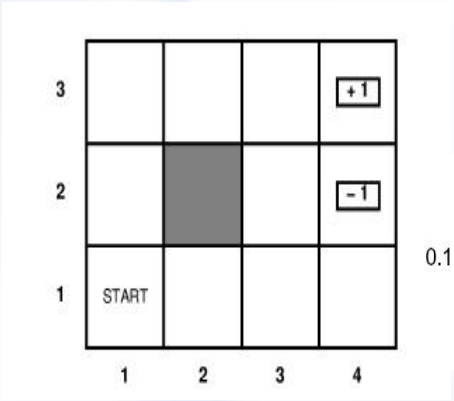
- **Naïve Bayesian Classifiers (NBC)**
- **Based on the simplifying assumption that the attribute values are conditionally independent given the target value:**

$$P(D|h) = P(d_1, \cdots, d_n|h) = \prod_i P(d_i|h)$$

- In discrete problems, learning task is to count the frequency of various data combinations within the training examples.

How to assume the function form?-4





$$s_0 \xrightarrow{a_0 : r_0} s_1 \xrightarrow{a_1 : r_1} s_2 \xrightarrow{a_2 : r_2} \cdots$$

- Transition model T(s,a,s'), how action influence states, e.g., T([1,1], up, [1,2]) = 0.8.
- Reward R(s), immediate value of state-action transition, e.g., r([4,3]) = +1.
- Policy $\pi(s)$ or $\pi(s,a)$, maps states to actions

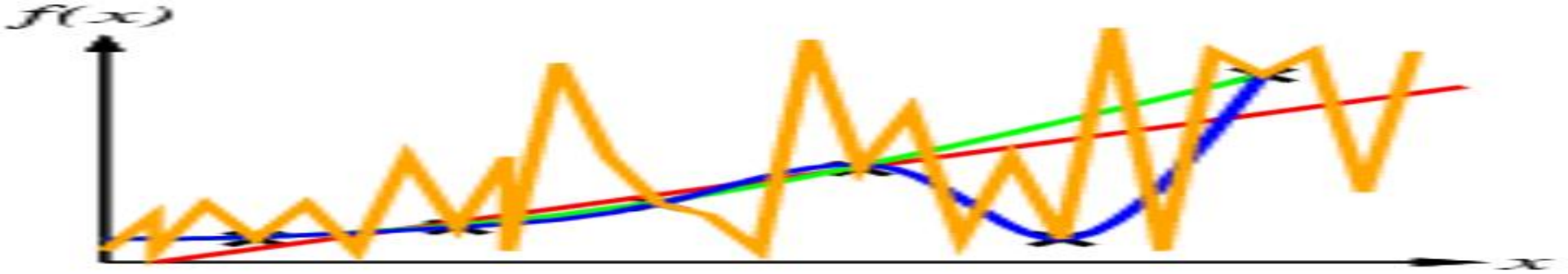
1.2 How to design the learning objective?

- Supervised Learning:

  - Minimize the error between the actual outputs and expected outputs for inputs (BP) (MSE)

  - Maximize Information Gain (IG) or reduction in entropy (Decision Tree)

  - Ockham's razor: prefer the simplest hypothesis consistent with data (Minimum Description Length, MDL)



How to design the learning objective?

- **Unsupervised Learning:**

  Minimize the distance between data and clusters (Clustering)

- **Reinforcement Learning:**

  Maximize cumulative reward in the long run

  Discounted Rewards

Road Map 2: Search

HOW TO FIND THE OPTIMAL MACHINES OR THE OPTIMAL SOLUTIONS TO THE PROBLEMS?



What is search ?

- With acceptable *costs*, find out an optimal or feasible *solution* in all the possible solutions of the problem
- Ideal search algorithm: find out the optimal solution as soon as possible.
- *Effectiveness* and *Efficiency* are contradictions to one another
  - *Completeness, Optimality, Complexity*
  - blind vs. heuristic, local vs. global,
  - and feasible vs. optimal.



Heuristic Search Methods

- **A central theme in mathematics and computer science**
- **Deterministic Search**
  - **Graph Search**
  - **Gradient Descent (Ascent), ID3**
  - **Q-learning**
- **Stochastic and Population based Search**
  - **Evolutionary Computation**
  - **Swarm Intelligence**

Key Problem in Population based Heuristic Search

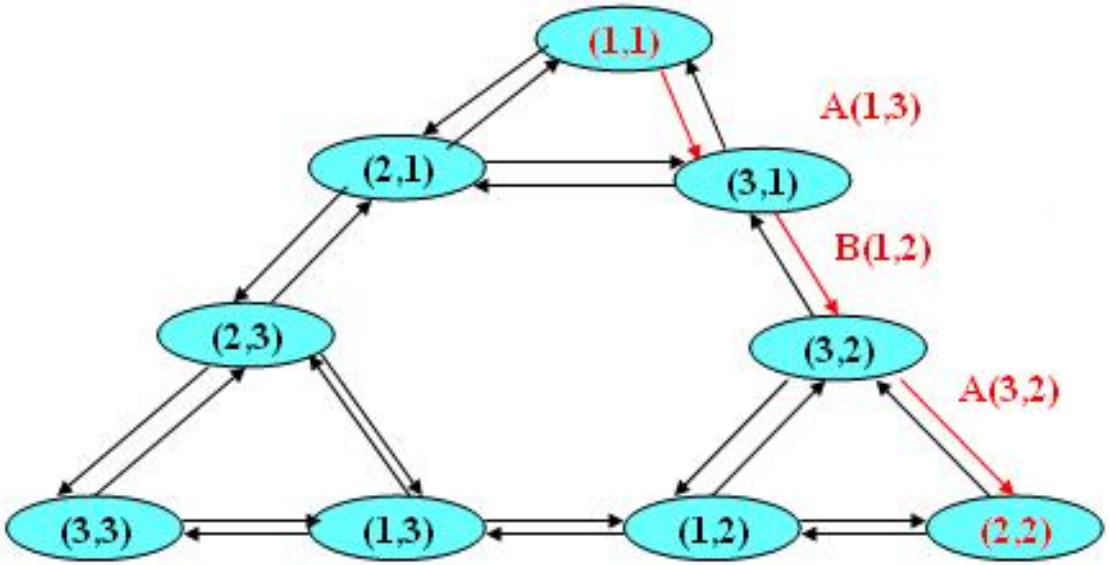
- **Exploitation/Exploration Balance**
  - **exploitation**
    reduce scope of search, try to improve the best-so-far individuals
  - **exploration**
    expand scope of search, sample unknown regions



2.1 Graph Search

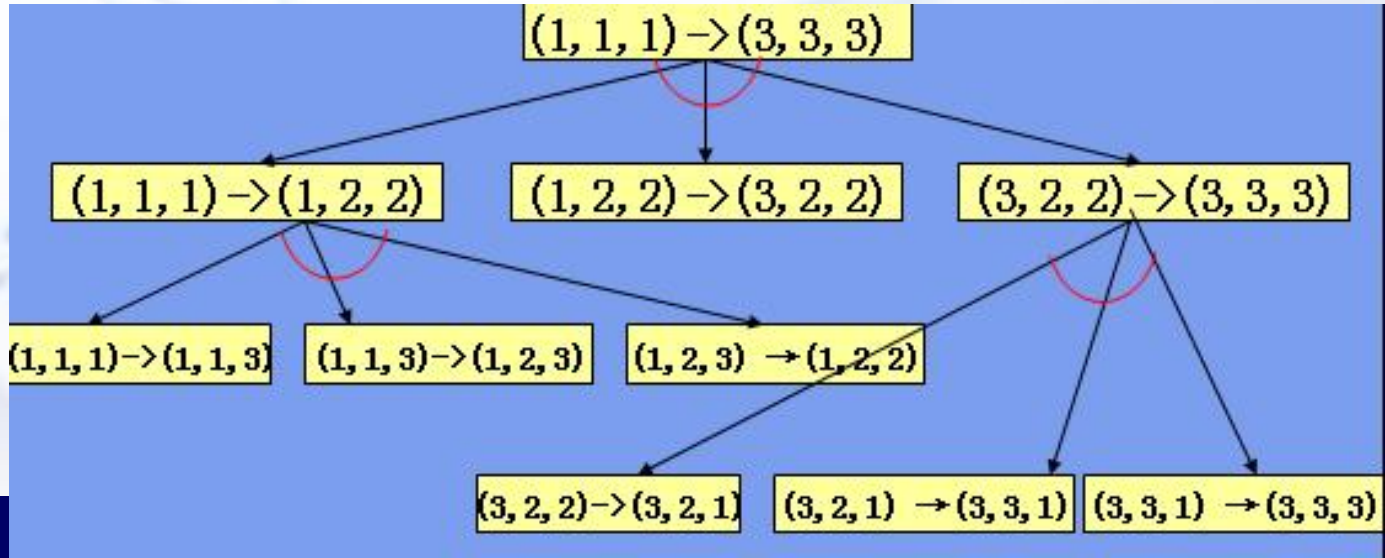
- For Problem-Solving
- Graph is gradually generated through expanding nodes in the search process until a goal state is found out (state space) or the initial node is labeled solvable or unsolvable (and/or graph).
- The key difference between different search strategies is in the order of node expanding.
- Blind search does not consider the particularity of the problem, which includes Bread-first search, Depth-first search, Depth-limited search and Iterative deepening search.
- Heuristic search employ the heuristic information provided by the problem to evaluate the costs of nodes. The node with lowest cost will be selected to expand.

State Space vs. And/Or Graph



State Space

And/Or Graph



A* algorithm

- **A* algorithm** is a well-known heuristic search algorithm for **state space**, which guarantee to find out the **optimal solution** through throwing a **constraint** on the heuristic function.
- **Admissible heuristics**: A heuristic $h(n)$ is **admissible** if for every node n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true** cost to reach the goal state from n.
- **Dominance**: If $h_2(n) \geq h_1(n)$ for all n (both admissible), then h_2 dominates h_1 , *and* h_2 is better for search

Game Search

- **Game tree**

  Data structure for representing game process

  = And/Or graph in structure when considering best achievable payoff against best play

- **Minimax algorithm**

  - Cut off the game tree for real decision-making

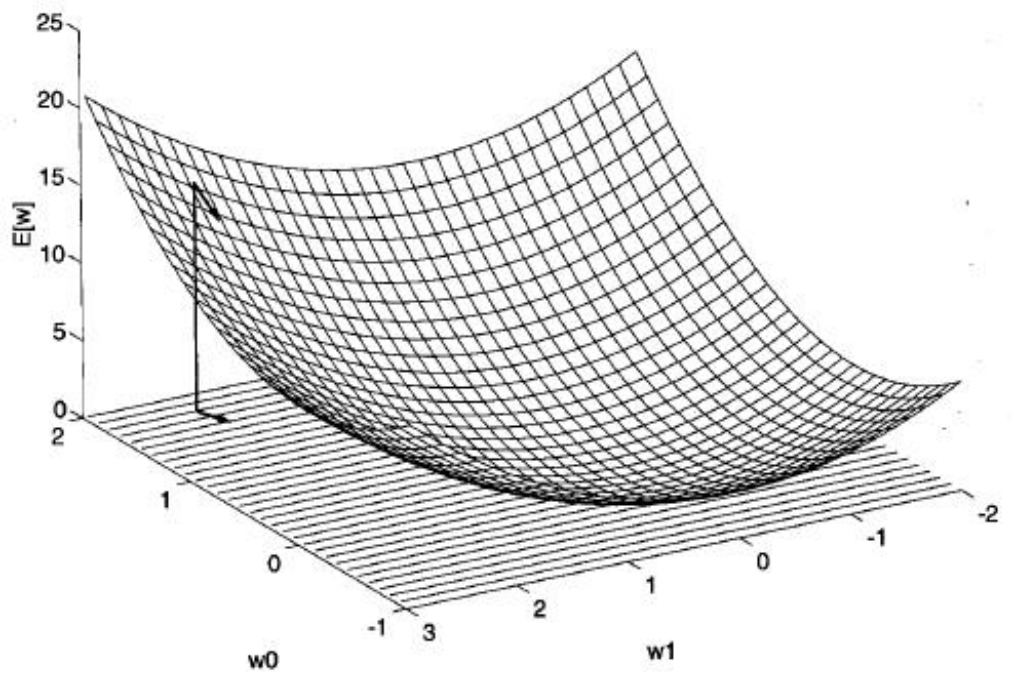
  - Evaluate the game state

  - choose move to position with highest minimax value!

- **α-β pruning**

  Improve search efficiency through pruning some unnecessary branches in the process of depth-limited search.

2.2 Gradient Descent Algorithm



$$\omega = \omega - \eta \frac{\partial e}{\partial \omega}$$

2.3 Decision Tree Learning (DTL) algorithm

```
function DTL(examples, attributes, default) returns a decision tree
    if examples is empty then return default
    else if all examples have the same classification then return the classification
    else if attributes is empty then return MODE(examples)
    else
        best ← CHOOSE-ATTRIBUTE(attributes, examples)
        tree ← a new decision tree with root test best
        for each value v_i of best do
            examples_i ← {elements of examples with best = v_i}
            subtree ← DTL(examples_i, attributes − best, MODE(examples))
            add a branch to tree with label v_i and subtree subtree
        return tree
```

Choose attribute based on information theory

- To implement `Choose-Attribute` in the DTL algorithm
- **Information Content (Entropy):**

$$I(P(v_1), \ldots, P(v_n)) = \Sigma_i -P(v_i) \log_2 P(v_i)$$

- For a training set containing *p* positive examples and *n* negative examples:

$$I(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

Information gain

- A chosen attribute *A* divides the training set *E* into subsets $E_1, \ldots, E_v$ according to their values for *A*, where *A* has v distinct values.

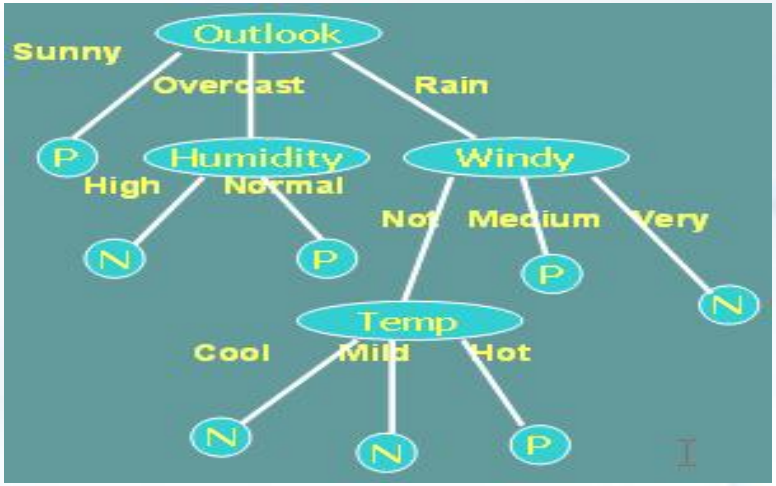
$$remainder(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

- **Information Gain (IG)** or reduction in entropy from the attribute test:

$$IG(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - remainder(A)$$

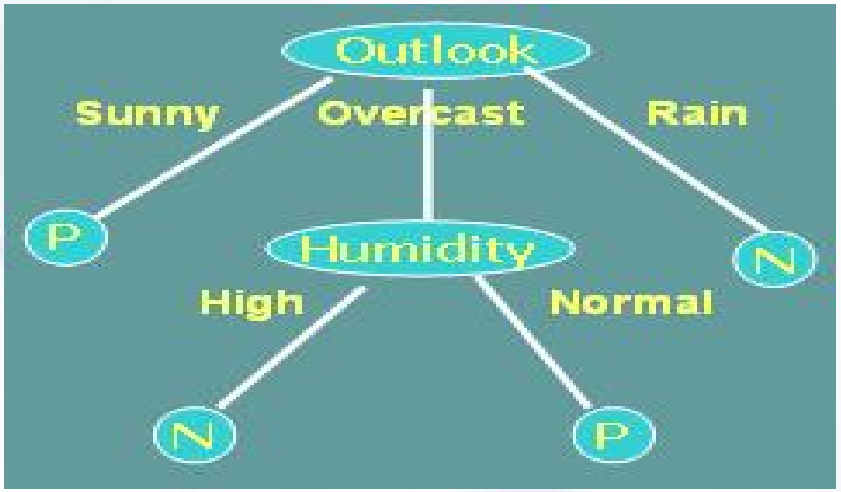
- Choose the attribute with the **largest IG**

MDL Criterion based DTL algorithm



With noise data 8

DL = 24.585 bits



Without noise data 8

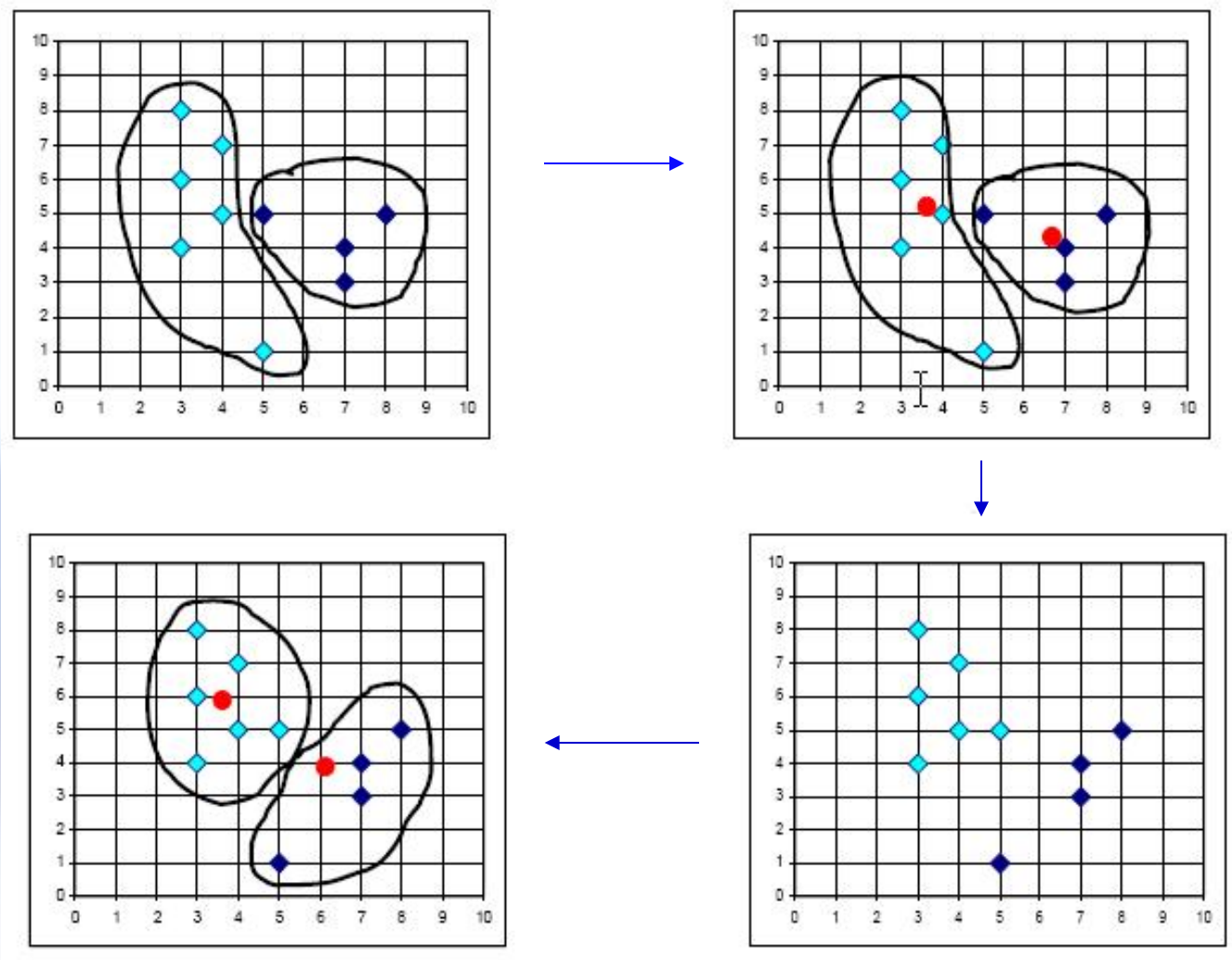
DL = 19.335

- Use the criterion of minimum DL increase, instead of maximum IG, to choose attributes in the process of generating the tree
- After the tree is generated, prune it at bottom-up mode until DL can not be reduced.

2.4 *k*-means & *k*-mediods algorithm

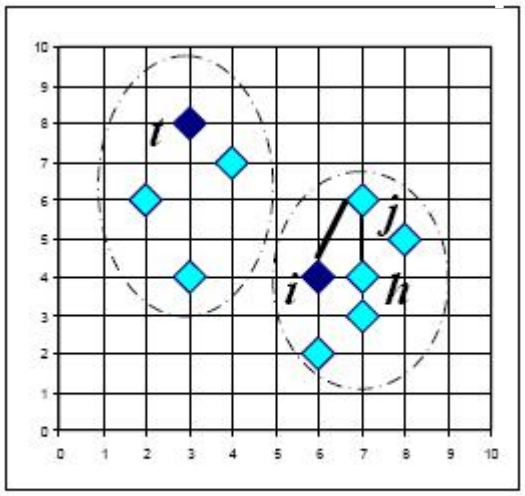
- ***k*-means algorithm is a main algorithm of partiton clustering, in which each cluster is represented by its mean and the data is partitioned into the closest cluster iteratively.**
- ***k*-means algorithm is changed into *k*-mediods algorithm if we represent each cluster using representative data.**

An example of K-means clustering

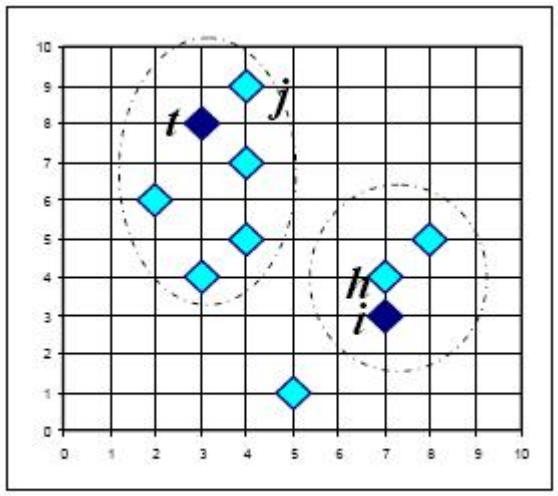


From “Data Mining: Concepts and Techniques”, J. Han and M. Kamber

An example of K-medoids clustering

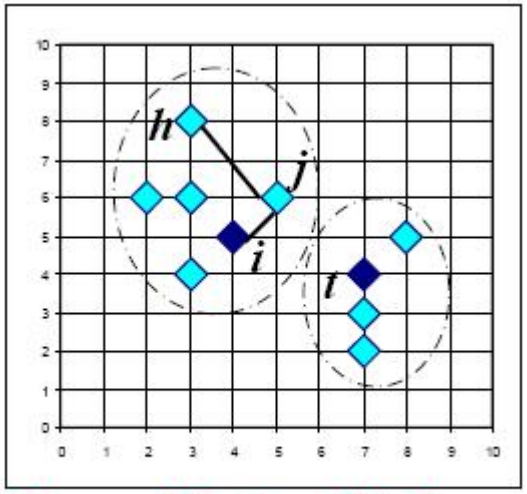


$C_{jih} = d(j, h) - d(j, i)$

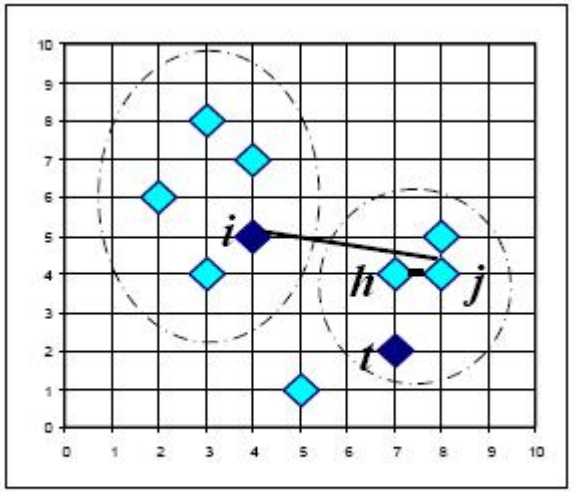


$C_{jih} = 0$

Substitution cost of each data



$C_{jih} = d(j, t) - d(j, i)$

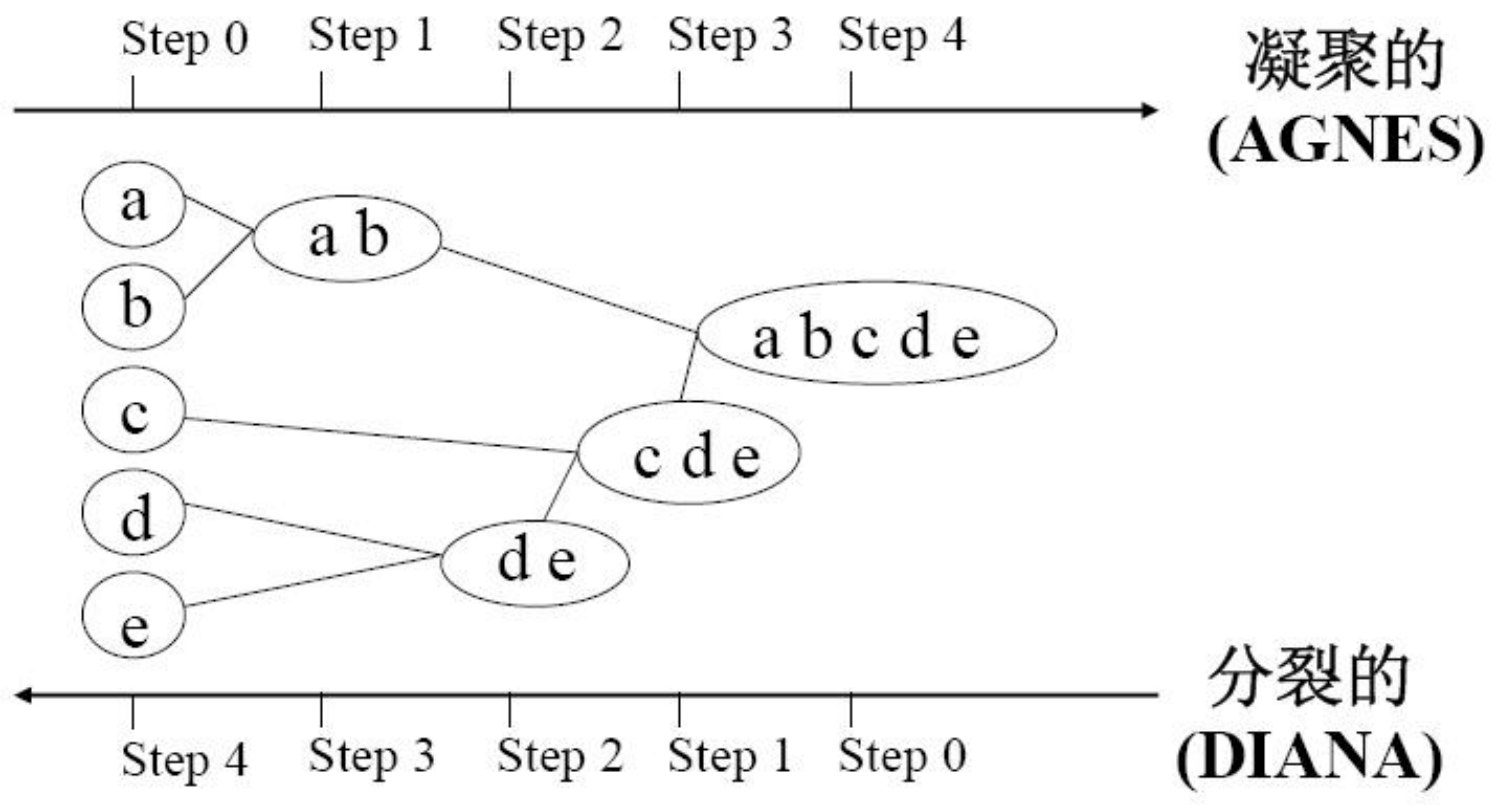


$C_{jih} = d(j, h) - d(j, t)$

Total substitution cost for the data set:

$$C_{ih} = \sum_j C_{jih}$$

2.5 Hierarchical clustering algorithms

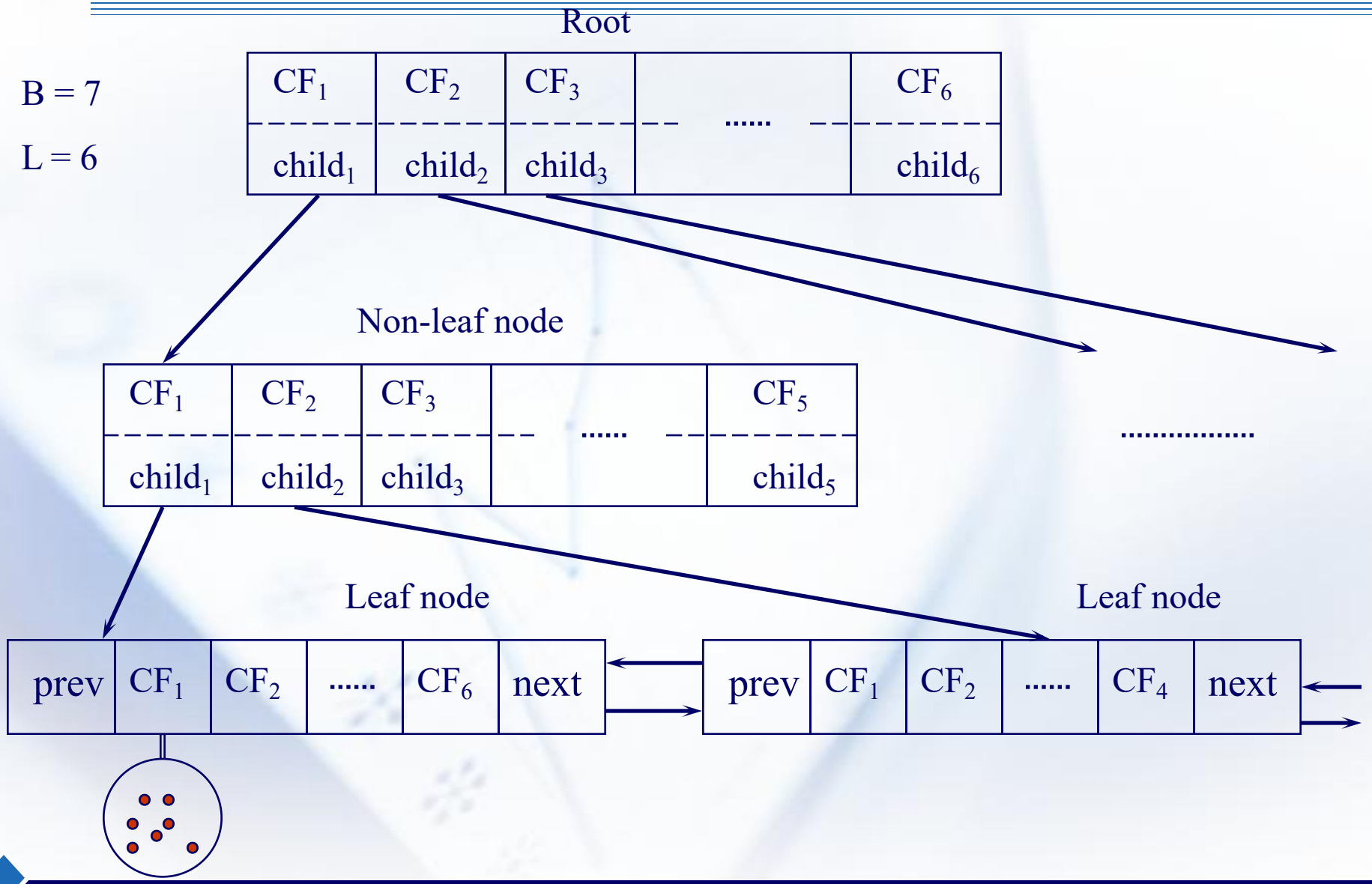


- **BIRCH (1996), CURE (1998), CHAMELEON (1999)**

BIRCH algorithm

- **Incrementally construct a CF (Clustering Feature) tree**
  - Parameters: max diameter (L), max children (B)
  - Phase 1: scan DB to build an initial in-memory CF tree (each node: #points, sum, sum of squares)
  - Phase 2: use an arbitrary clustering algorithm to cluster the leaf nodes of the CF-tree
- ***Scales linearly*: finds a good clustering with a single scan and improves the quality with a few additional scans**
- ***Weaknesses:* handles only numeric data, sensitive to order of data records.**

CF Tree Example



CURE algorithm

Step1. Sample the data set，get the sample set

Step2. Partition the sample set into *p* parts, each of which include *n*/*p* samples.

Step3. Perform CURE hierarchal clustering on each part, the cluster number is set to be *n*/*pq*

Step4. Delete singular points (very small cluster)

Step5. Perform CURE hierarchal clustering on generated clusters.

Step6. Assign each data into the closest cluster.

Core: CURE hierarchal clustering

```
procedure cluster(S, k)
begin
1.  T := build_kd_tree(S)
2.  Q := build_heap(S)
3.  while size(Q) > k do {
4.      u := extract_min(Q)
5.      v := u.closest
6.      delete(Q, v)
7.      w := merge(u, v)
8.      delete_rep(T, u); delete_rep(T, v); insert_rep(T, w)
9.      w.closest := x /* x is an arbitrary cluster in Q */
10.     for each x ∈ Q do {
11.         if dist(w, x) < dist(w, w.closest)
12.             w.closest := x
13.         if x.closest is either u or v {
14.             if dist(x, x.closest) < dist(x, w)
15.                 x.closest := closest_cluster(T, x, dist(x, w))
16.             else
17.                 x.closest := w
18.             relocate(Q, x)
19.         }
20.         else if dist(x, x.closest) > dist(x, w) {
21.             x.closest := w
22.             relocate(Q, x)
23.         }
24.     }
25.     insert(Q, w)
26. }
end
```

Until k clusters

Determine the clusters for merge

Perform merge

Recompute the closest cluster

```
procedure merge(u, v)
begin
1.  w := u ∪ v
2.  w.mean := (|u|u.mean+|v|v.mean) / (|u|+|v|)
3.  tmpSet := ∅
4.  for i := 1 to c do {
5.      maxDist := 0
6.      foreach point p in cluster w do {
7.          if i = 1
8.              minDist := dist(p, w.mean)
9.          else
10.             minDist := min{dist(p, q) : q ∈ tmpSet}
11.         if (minDist ≥ maxDist){
12.             maxDist := minDist
13.             maxPoint := p
14.         }
15.     }
16.     tmpSet := tmpSet ∪ {maxPoint}
17. }
18. foreach point p in tmpSet do
19.     w.rep := w.rep ∪ {p + α*(w.mean-p) }
20. return w
end
```

Select representative points

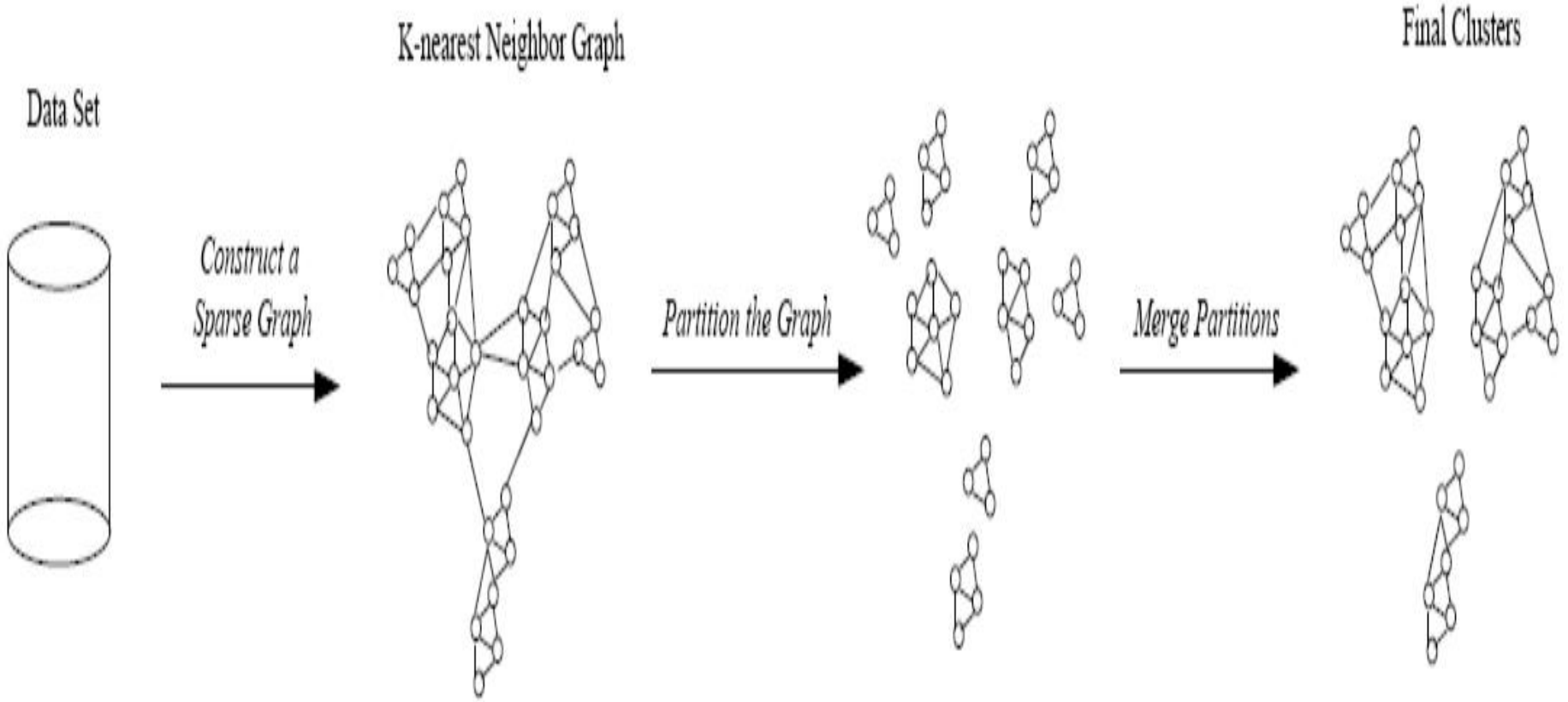
Point shrunk

CHAMELEON algorithm

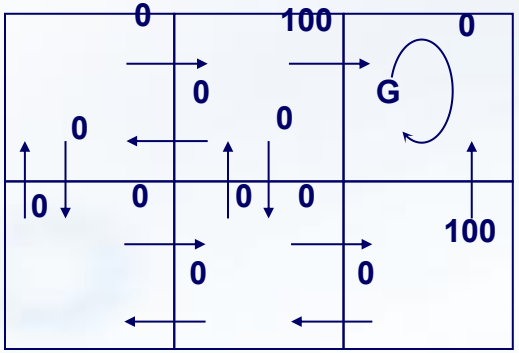
- **A hierarchical Clustering Algorithm Using Dynamic Modeling**
- **Two-phase approach**
- **Phase –I (similarity between points)**
  - Uses a graph partitioning algorithm to divide the data set into a set of individual clusters.
- **Phase -II (similarity between clusters)**
  - uses an agglomerative hierarchical mining algorithm to merge the clusters.

Graph based Method

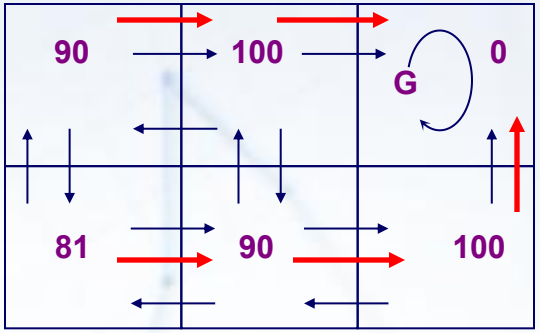
CHAMELEON Steps



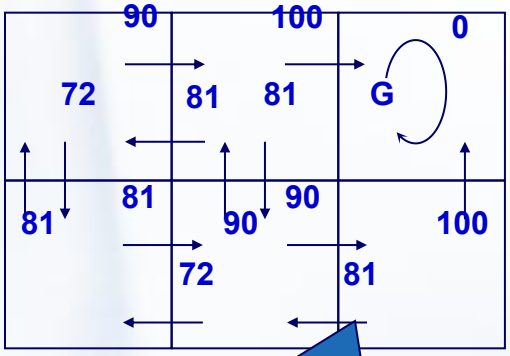
2.6 Q-Learning Algorithm



***r*(*state*, *action*) immediate reward values**



V(**state**) values*



***Q*(*st*... ...lues**

81=0+0.9*90

$$V^*(s_t) = \max_{\pi} V^{\pi}(s_t), \forall s_t \in \boldsymbol{S}$$

$$V^{\pi}(s_t) = \sum_{i=0}^{\infty} \gamma^i r_{t+i}$$

$$Q(s_t, a) = f_R(s_t, a) + \gamma V^*(f_S(s_t, a))$$

- Using the *discounted cumulative reward*, the discount factor = 0.9

Learning the Q-value

- **Note: *Q* and *V** closely related**

$$V^*(s) \equiv \arg\max_{a'} Q(s, a') \qquad Q(s_t, a) = f_R(s_t, a) + \gamma V^*(f_S(s_t, a))$$

- **Allows us to write *Q* recursively as**

$$Q(s_t, a) = f_R(s_t, a) + \gamma \max_{a'} Q(f_S(s_t, a), a')$$

Solve: How to learn?

- **Using Q-values**

$$\pi^*(s_t) = \arg\max_a Q(s_t, a)$$

Solve: How to choose the best action?

A kind of Temporal Difference (TD) learning

Q-Learning Steps

- **FOR each <s, a> DO**
  - *Initialize* table entry: $\hat{Q}(s, a) \leftarrow 0$
- **Observe current state s**
- **WHILE (true) DO**
  - *Select* action a and execute it
  - *Receive* immediate reward r
  - *Observe* new state s'
  - *Update* table entry for $\hat{Q}(s, a)$ as follows

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a'} \hat{Q}(s', a')$$

  - *Move*: record transition from s to s'

2.7 Evolutionary Computation

- **Evolutionary Computation is based on biological metaphor:**
  - Optimized Behavior – Optimal Solution
  - Individual – Solution; Fitness – Objective; Environment – Problem
- **Ingredients of EC**
  - Representation: Individual and Population
  - Fitness evaluation: Objective
  - Selection: Parent and Survivor
  - Genetic Operators: Mutation and/or Recombination
  - Initialization/Termination

The Ingredients

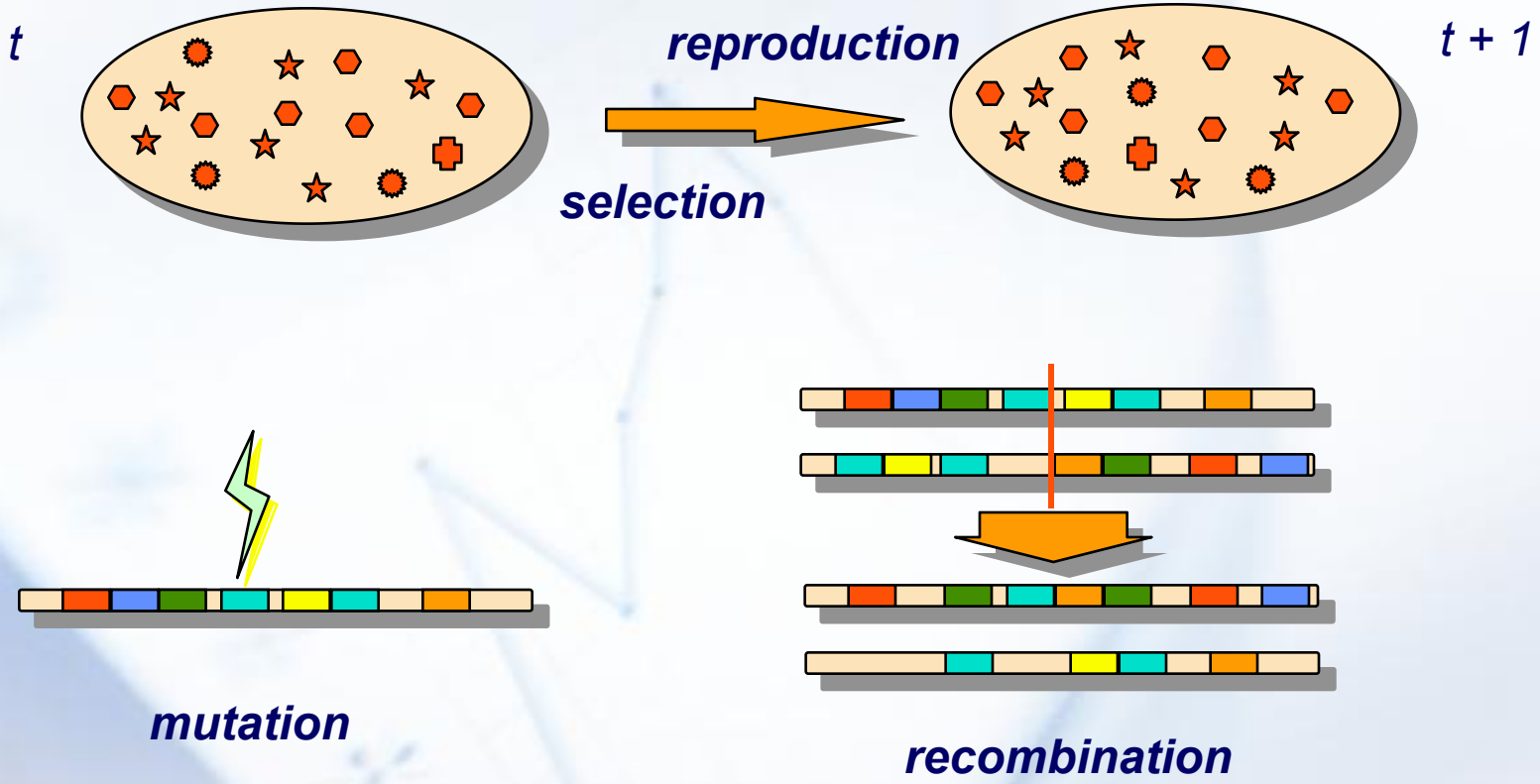


Image from Ida Sprinkhuizen-Kuyper: Introduction to Evolutionary Computation, 2000.

The Evolution Mechanism

2026-10-18

- **Increasing diversity by genetic operators**
  - mutation
  - Recombination
- **Decreasing diversity by selection**
  - of parents
  - of survivors

The Evolutionary Cycle

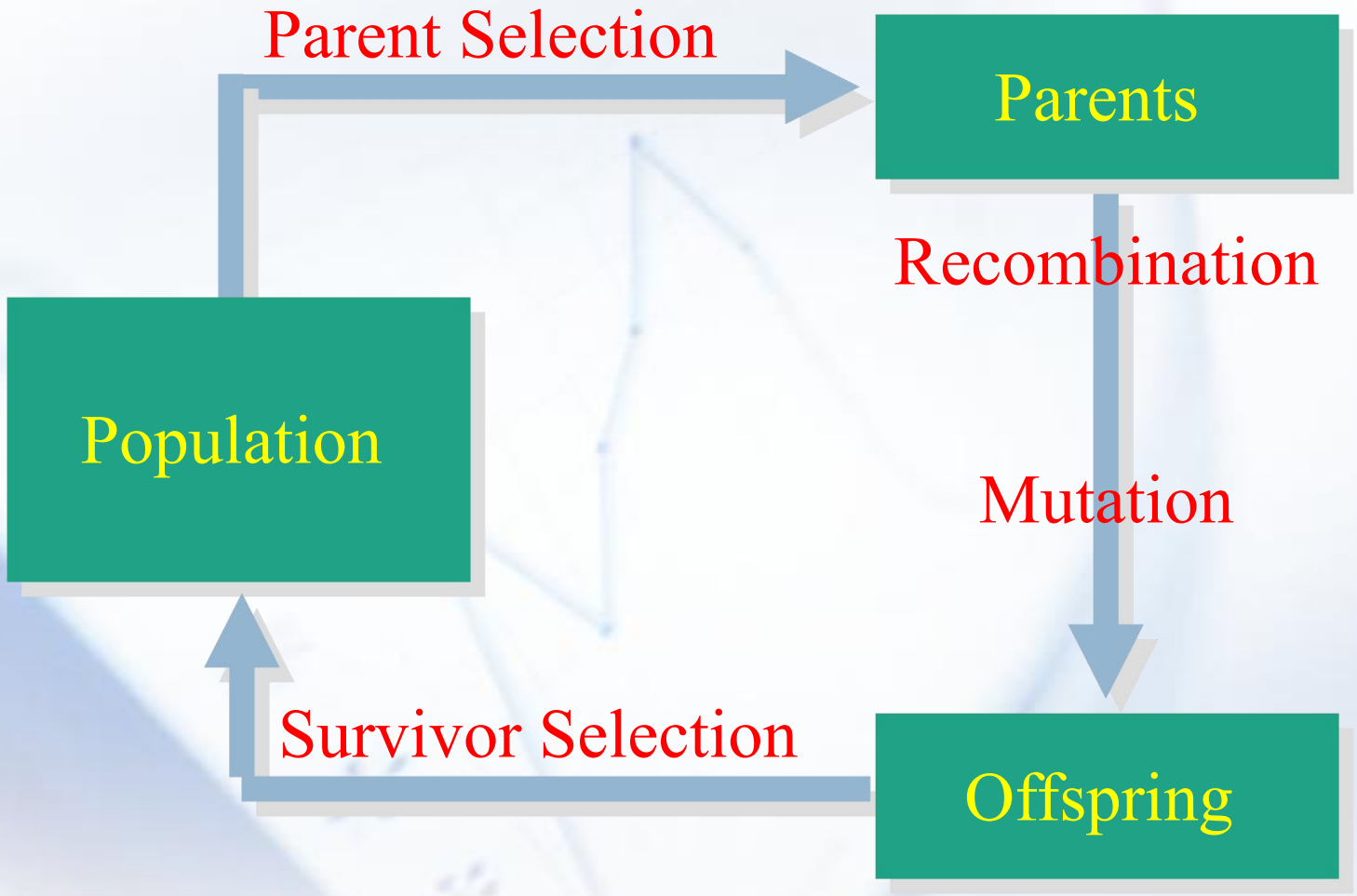


Image from Ben Paechter: Evolutionary Computing – A Practical Introduction

Desirable Performance of EC

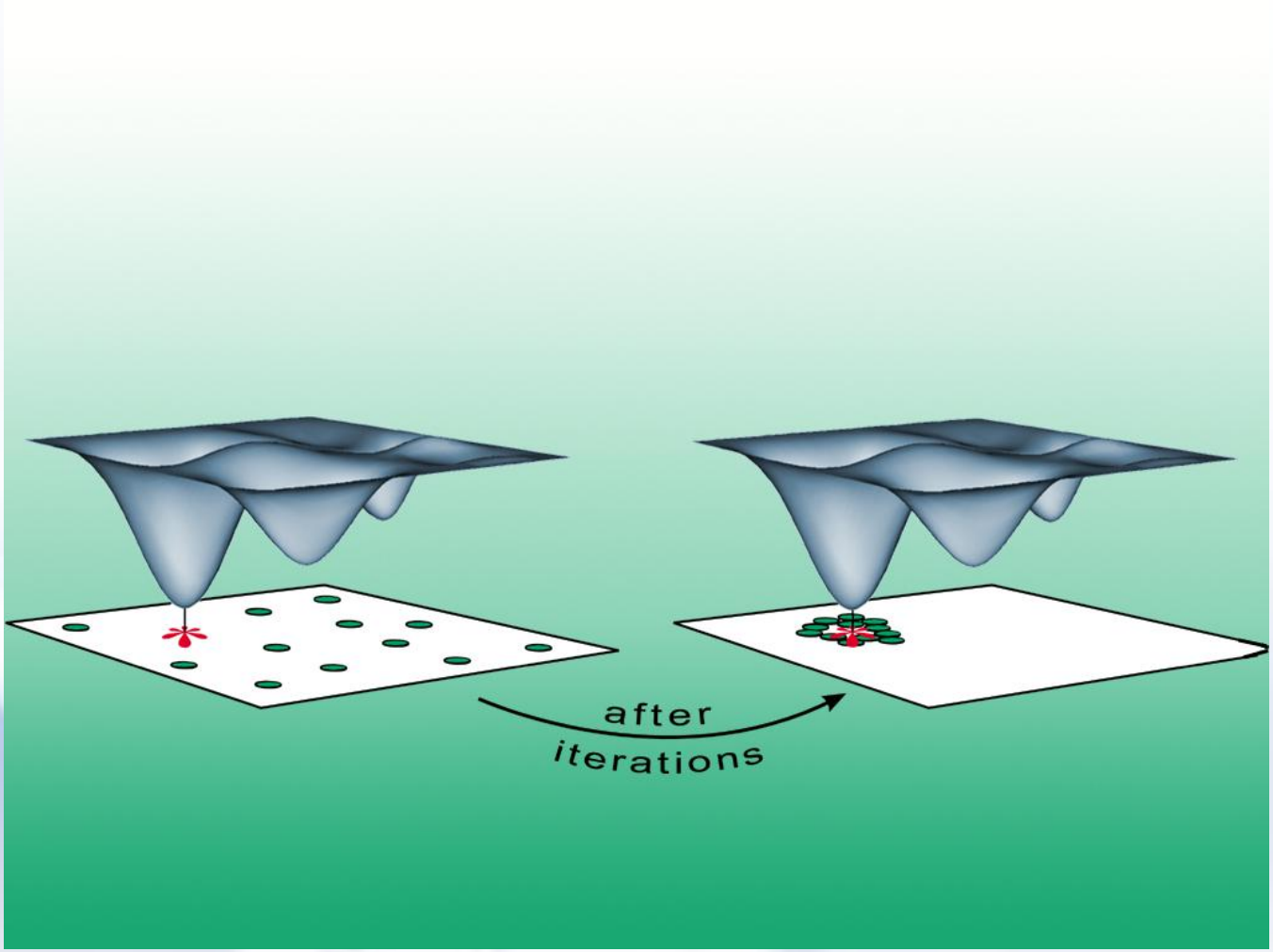


Image from: Introduction to Stochastic Search and Optimization (ISSO) by J. C. Spall

How to build a EA?

Step1. Design a representation

Step2. Design a way of mapping a genotype to a phenotype (not necessarily)

Step3. Design a fitness function

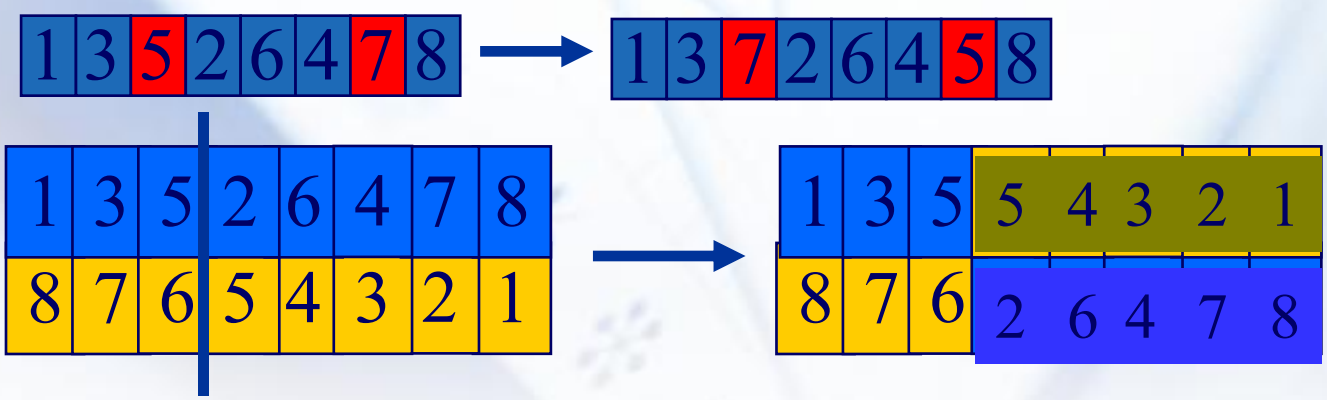
Step4. Design suitable genetic operators: mutation and/or recombination

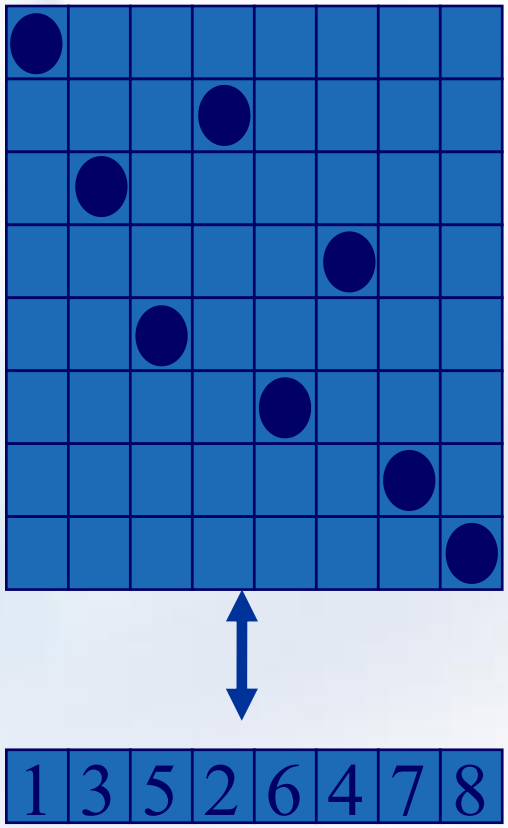
Step5. Decide how to select parents and survivors

Step6. Decide how to initialise a population and when to stop the algorithm

Example EA for 8 Queens Problem

Representation	Permutations
Recombination	"Cut-and-crossfill" crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	80%
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation



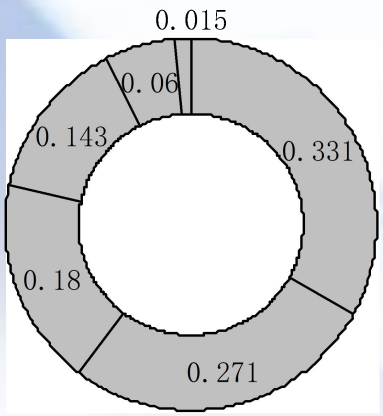


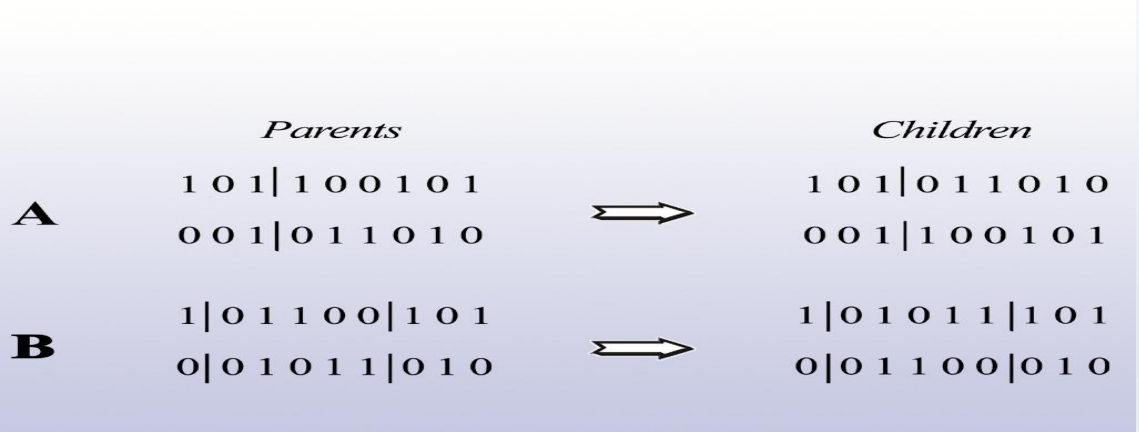
Features and Main Streams of EC

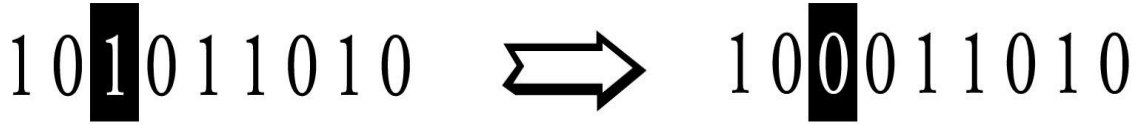
- **Main Features of EC:**
  - Parallel and Stochastic
  - Easy applicable
- **Main streams of Evolutionary Algorithms**
  - Genetic Algorithm
  - Evolutionary Programming
  - Evolutionary Strategies
  - Genetic Programming

Genetic Algorithm

- *Representation*: Bit-string (Genotype)
- *Mutation*: Bit-swap
- *Recombination*: Crossover
- *Parent Selection*: Chance according to Fitness
- *Survivor Selection*: Extinct Replacement

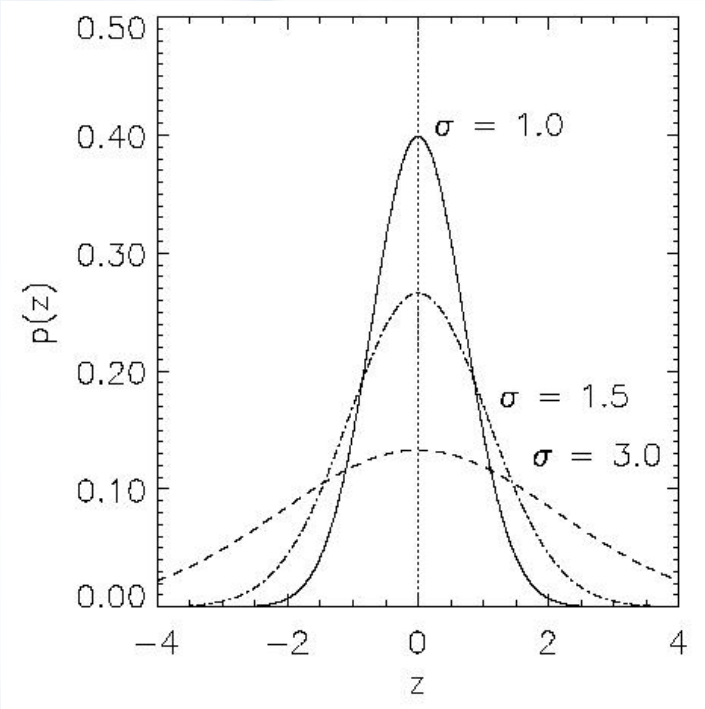






Evolutionary Programming and Evolutionary Strategies

- *Representation*: Real Value (Phenotype)
- *Mutation*: Gaussian Disturb
- *Recombination*: None! (EP) vs. Crossover (ES)
- *Parent Selection*: Deterministic (EP) vs. Uniform Distribution (ES)
- *Survivor Selection*: Tournament (EP) vs. Deterministic (ES)



How can we train the game player using NN and EC?

- **Neural nets for evaluating future values of moves are evolved**
- **NNs have fixed structure with 5046 weights, these are evolved + one weight for "kings"**
- **Representation:**
  - vector of 5046 real numbers for object variables (weights)
  - vector of 5046 real numbers for σ's
- **Mutation:**
  - Gaussian, lognormal scheme with σ-first
  - Plus special mechanism for the kings' weight

Example from [Fogel and Chellapilla, 2002]

2.8 Swarm Intelligence

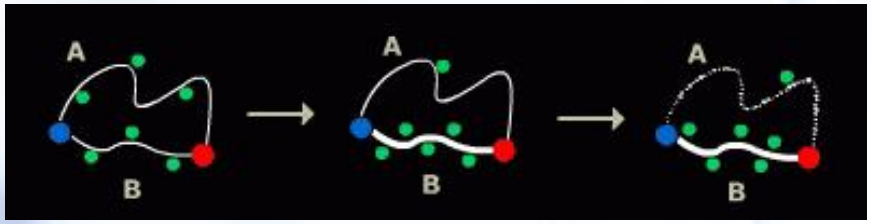
- **Swarm Intelligence (SI)**
  - an artificial intelligence technique based around the study of **collective behavior** in **decentralized, self-organized** systems.
- **Multi-Agent Systems (MAS)**
  - A system that consists of a number of agents, which **interact** with one-another.
  - **Communication, Coordination, Collaboration**

Swarm Intelligence for Search

- **Ant Colony Optimization (ACO)**
  - Inspired by **ant colony foraging**
  - **pheromone** as heuristic information (stigmergy)
  - Iteration between **ConstructAntSolutions** and **UpdatePheromones**
- **Particle Swarm Optimization (PSO)**
  - Inspired by **bird flocking**
  - Heuristic information: **results from partners**
  - **Particle Velocity Update**

1）Ant Colony Optimization (ACO)





- Since the route B is shorter, the ants on this path will complete the travel more times and thereby lay more pheromone over it.

- The pheromone concentration on trail B will increase at a higher rate than on A, and soon the ants on route A will choose to follow route B
- Since most ants will no longer travel on route A, and since the pheromone is volatile, trail A will start evaporating
- Only the shortest route will remain!

Model Ant Colony for Optimization

- Each **artificial ant** is a probabilistic mechanism that constructs a solution to the problem, using:
  - Artificial pheromone deposition
  - Heuristic information: pheromone trails, and others

- **Differences** between real and artificial ants:
  - The pheromone is updated only after a solution has been constructed.
  - Additional mechanisms

Algorithm 1 Ant colony optimization metaheuristic

```
Set parameters, initialize pheromone trails
while termination conditions not met do
  ConstructAntSolutions
  ApplyLocalSearch        {optional}
  UpdatePheromones
end while
```

Model Ant Colony for Optimization

ConstructAntSolutions

- The rules for the **probabilistic choice of solution components**.

UpdatePheromones

- It is used to **increase** the pheromone values associated with good or promising solutions, and **decrease** those that are associated with bad ones.
  - Decreasing all the pheromone values through **pheromone evaporation** --> allows “forgetting”-> favors exploration of new areas
  - Increasing the pheromone levels associated with a **chosen set** of good solutions -> makes the algorithm converge to a solution

Trivial Example of TSP (4 Cities)

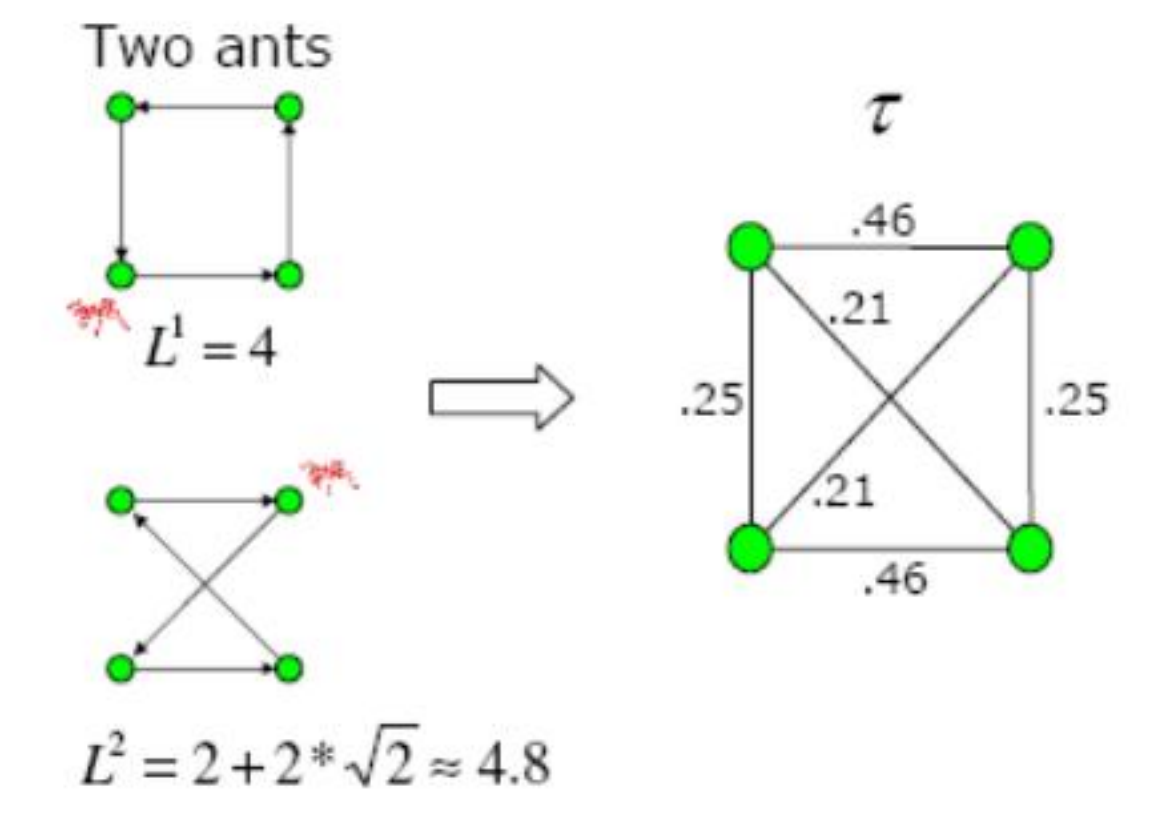
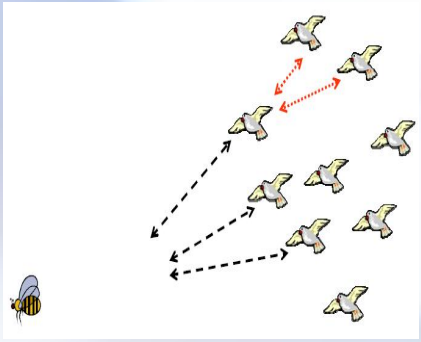
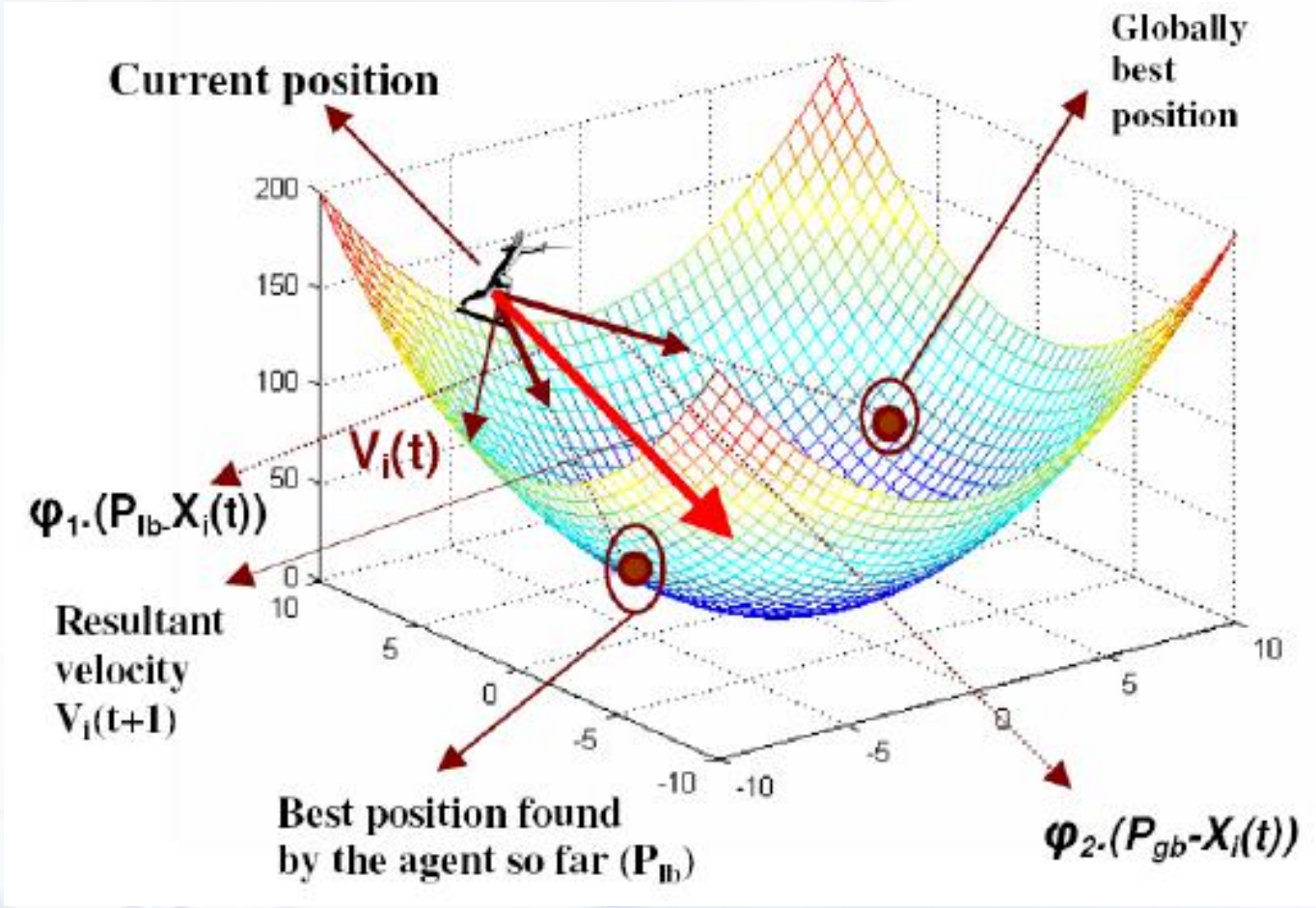


Image from Olle Gallmo: Swarm Intelligence

2）Particle fly to find the best solution





How do particles fly?

- **Combination of gBest and the pBest (lBest)**
  - need a compromise
- **lBest can be:**
  - **Social**: the particles around are always the same, no matter where they are in space
  - **Geographical**: the particles around are those whose distance is the shortest
- **Global PSO vs. Local PSO**
  - the global version converges quickly to a solution but it gets more easily stuck in local minima.

PSO Velocity Update Equations

❖ Global Version:

$$v_{ik}^{t+1} = \omega v_{ik}^{t} + c_1 \xi \left(p_{ik}^{t} - x_{ik}^{t} \right) + c_2 \eta \left(p_{gk}^{t} - x_{ik}^{t} \right), k = 1,2,\cdots,d$$

$$x_{ik}^{t+1} = x_{ik}^{t} + v_{ik}^{t+1}, k = 1,2,\cdots,d$$

where ***k*** is the dimension, c_1 and c_2 are positive constants, ξ and η are random functions, and ***w*** is the inertia weight.

For neighborhood version, change p_{gk} to p_{lk}.

Particle Swarm Optimization Process

Step1. Initialize population in hyperspace.

Step2. Evaluate fitness of individual particles.

Step3. Modify velocities based on previous best and global (or neighborhood) best.

Step4. Terminate on some condition.

Step5. Go to step 2.

Road Map 3: Reasoning

3.1 Knowledge Representation

- **Production rule and system**

  condition-action rules

  system structure: working memory, rule base and interpreter

  working model: match-resolve-act cycle

- **Propositional logic and first-order logic**

  Sentences: Constants, Predicates, Functions, Variables, Connectives, Quantifiers

3.2 Production Rule

P → Q

or IF P THEN Q (certainty factor)

Antecedent
Condition

Consequent
Conclusion or Action

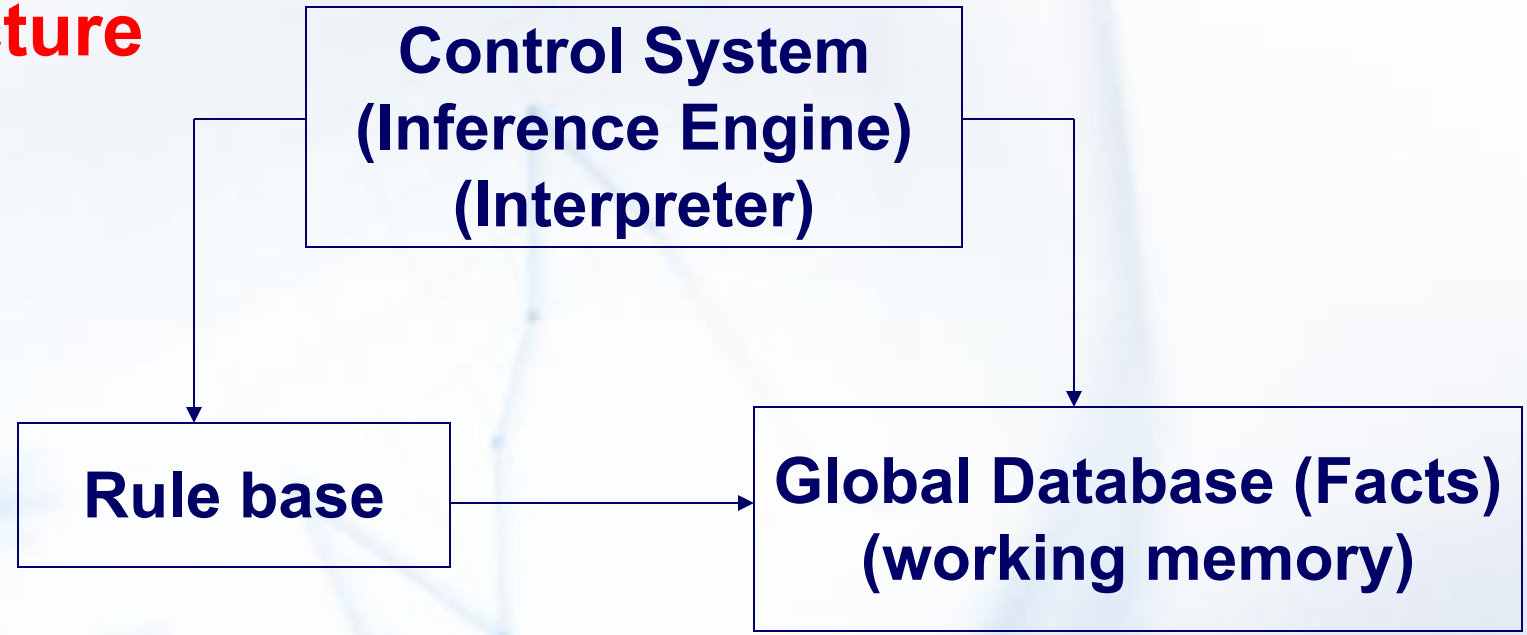
E.g.,

IF *An animal has teeth, claw, and eyes staring into the frontage*

Then *the animal is carnivore* (0.6)

Production-rule System

- **Structure**



- **Procedure**

Rule Matching → Conflict Resolution → Rule Execution

Stop Conditions

3.3 Syntax of First-Order Logic (FOL): Basic elements

- **Constants** E.g., John, 2, Sun,...
- **Functions** E.g., Sqrt, FatherOf,...
- **Variables** E.g., x, y, a, b,...

} Term

- **Predicates** E.g., Brother, >,...
- **Connectives** $\neg, \rightarrow, \wedge, \vee, \leftrightarrow$
- **Quantifiers** $\forall, \exists$

First-Order Logic (FOL)

- **First-order inference rules** are fundaments in first-order inference
  - **Validity and Satifiability**
  - **Equivalence and Entailment**
- **Unification** is necessary in first-order inference
  - **Most general unification**

Resolution Deduction

- **Resolution is a kind of reduction to absurdity, which prove *facts* => *conclusion* by proving *facts*∧¬*conclusion* unsatisfiable, i.e. by deriving the empty clause**

  conjunctive normal form

  resolution strategies

Resolution Rule

- **Eliminate complementary literals in two sentences, the disjunction of remaining parts of two sentences construct the resolvent clause which is the entailment of two sentences above.**

  Literals are complementary if one is the negation of the other.

- **Apply resolution steps to CNF(KB $\wedge$ $\neg\alpha$); complete for inference when empty clause is reached.**

Conclusions

Course Achievement

Have you ever enjoyed your learning of AI? Do you feel the charm of AI and wish to know it more?

Have you got overall understanding of AI, and prepared yourself for solving the related problems in the future work?

Course Grading

Final exam: 40%

Term projects: 50%

Participation in the class and Exercises: 10%

Bonus: at most 12 scores

Questions?

E_mail: liuxiabi@bit.edu.cn

End of the Course

See you !